Tutorial: gene- and transcript-level analysis of the airway data

Lieven Clement

```
library(tidyverse)
library(SummarizedExperiment)
library(edgeR)
library(tximeta)
library(org.Hs.eg.db)
library(stageR)
library(patchwork)
```

In this tutorial we apply the workflows of the introduction to bulk RNA-seq and the technical details and transcript usage lectures to a new dataset. We analyse the same experiment at three levels:

1. **Differential gene expression (DGE)**: does the expression of a gene change?
2. **Differential transcript expression (DTE)**: does the expression of a transcript change?
3. **Differential transcript usage (DTU)**: does the relative contribution of the transcripts to the expression of their gene change?

Background

Glucocorticoids are anti-inflammatory drugs that are used, for example, by people with asthma to reduce the inflammation of the airways. Himes et al. (2014) studied the effect of the synthetic glucocorticoid dexamethasone on human airway smooth muscle cells. Cells of four cell lines were treated with 1 µM dexamethasone for 18 hours or left untreated. The data are available in GEO under accession number GSE52778.

The paired-end reads of the 8 samples were quantified with salmon against the GENCODE 29 human transcriptome, with 20 Gibbs samples per sample to quantify the uncertainty of the transcript counts:

```
salmon index -t gencode.v29.transcripts.fa.gz -i salmon_gencode29_k31 -k 31 --gencode
salmon quant -i salmon_gencode29_k31 -l A \
  -1 SRR1039508_1.fastq.gz -2 SRR1039508_2.fastq.gz \
  --gcBias --seqBias --numGibbsSamples 20 -o quants/SRR1039508
```

Data import

The salmon output is available as a release of the course repository (215 MB). We download and unzip it once and then remove the zip file; the default time-out of `download.file` (60 seconds) is too short for a file of this size.

```
zipUrl <- "https://github.com/statOmics/SGA/releases/download/airway-salmon-v1/airwaySalmon_g
if (!dir.exists("airwaySalmon")) {
  options(timeout = 1200)
  download.file(zipUrl, "airwaySalmon.zip", mode = "wb")
  unzip("airwaySalmon.zip")
  unlink("airwaySalmon.zip")
}
```

```
coldata <- read.csv("airwaySalmon/coldata.csv") |>
  mutate(
    cell = factor(cell),
    dex = factor(dex, levels = c("untrt", "trt")),
    files = file.path("airwaySalmon/quants", names, "quant.sf.gz")
  )
coldata |>
  dplyr::select(-files)
```

```
       names    cell   dex
1 SRR1039508  N61311 untrt
2 SRR1039509  N61311   trt
3 SRR1039512 N052611 untrt
4 SRR1039513 N052611   trt
5 SRR1039516 N080611 untrt
6 SRR1039517 N080611   trt
7 SRR1039520 N061011 untrt
8 SRR1039521 N061011   trt
```

Question

- What is the experimental unit?
- Which type of design is this? Which model do you propose?

- Every cell line contributes an untreated and a treated sample: the cell lines are **blocks** and the design is a randomised complete block design, as the donors in the macrophage data.
- We correct for the cell line in the model, so that the treatment effect is estimated within each cell line:

```
design <- model.matrix(~ cell + dex, data = coldata)
colnames(design)
```

```
[1] "(Intercept)" "cellN061011" "cellN080611" "cellN61311"  "dextrt"
```

tximeta imports the salmon output. It recognises the GENCODE 29 transcriptome from the checksum of the index, adds the transcript annotation and imports the Gibbs samples as additional assays.

```
st <- tximeta(coldata)
st
```

```
class: RangedSummarizedExperiment
dim: 205870 8
metadata(6): tximetaInfo quantInfo ... txomeInfo txdbInfo
assays(23): counts abundance ... infRep19 infRep20
rownames(205870): ENST00000456328.2 ENST00000450305.2 ...
  ENST00000387460.2 ENST00000387461.2
rowData names(3): tx_id gene_id tx_name
colnames(8): SRR1039508 SRR1039509 ... SRR1039520 SRR1039521
colData names(3): names cell dex
```

```
assayNames(st) |> head()
```

```
[1] "counts"    "abundance" "length"    "infRep1"   "infRep2"   "infRep3"
```

```
metadata(st)$quantInfo$percent_mapped
```

```
[1] 94.35475 93.71777 95.01937 95.54629 94.53346 94.98765 94.87319 95.23916
```

Part 1: differential gene expression

Gene-level data

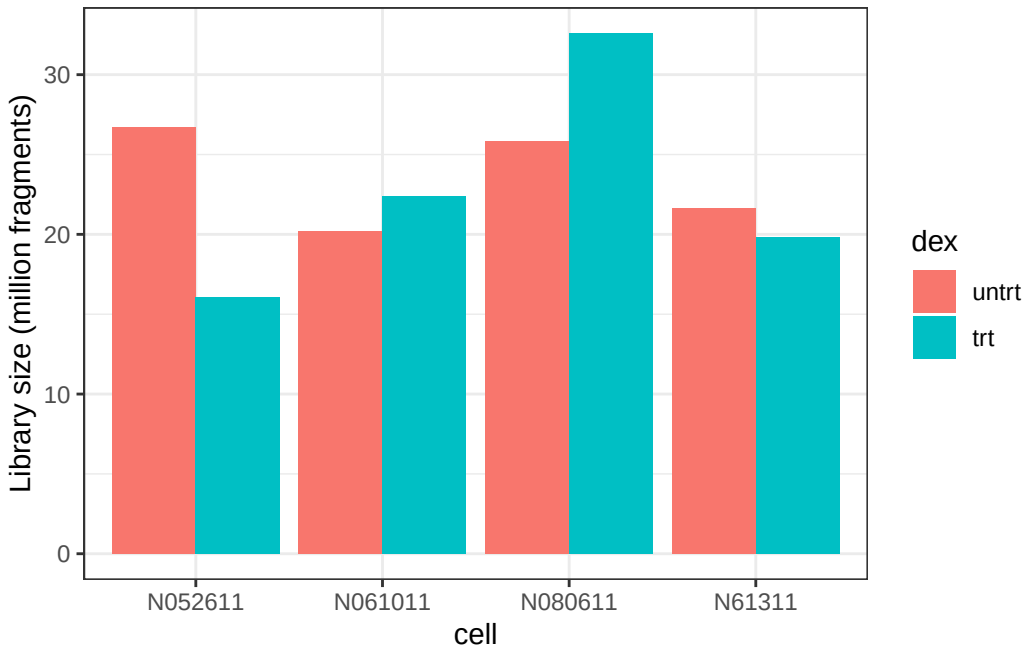
We summarise the transcript-level counts to the gene level and add the gene symbols.

```
sg <- summarizeToGene(st)
sg <- addIds(sg, "SYMBOL", gene = TRUE)
```

`DGEListFromTximeta` creates the `DGEList` and stores the centred log average transcript lengths as offsets (see the introduction).

```
dge <- DGEListFromTximeta(sg)
dge$samples <- cbind(dge$samples, coldata |> dplyr::select(cell, dex))
```

```
dge$samples |>
  ggplot(aes(cell, lib.size / 1e6, fill = dex)) +
  geom_col(position = "dodge") +
  ylab("Library size (million fragments)") +
  theme_bw()
```



> Question
>
> How do the library sizes differ between the samples? Is this a problem?

- The library sizes range from 16 to 33 million fragments.
- They are not related to the treatment, and the differences in sequencing depth are accounted for by the offsets.

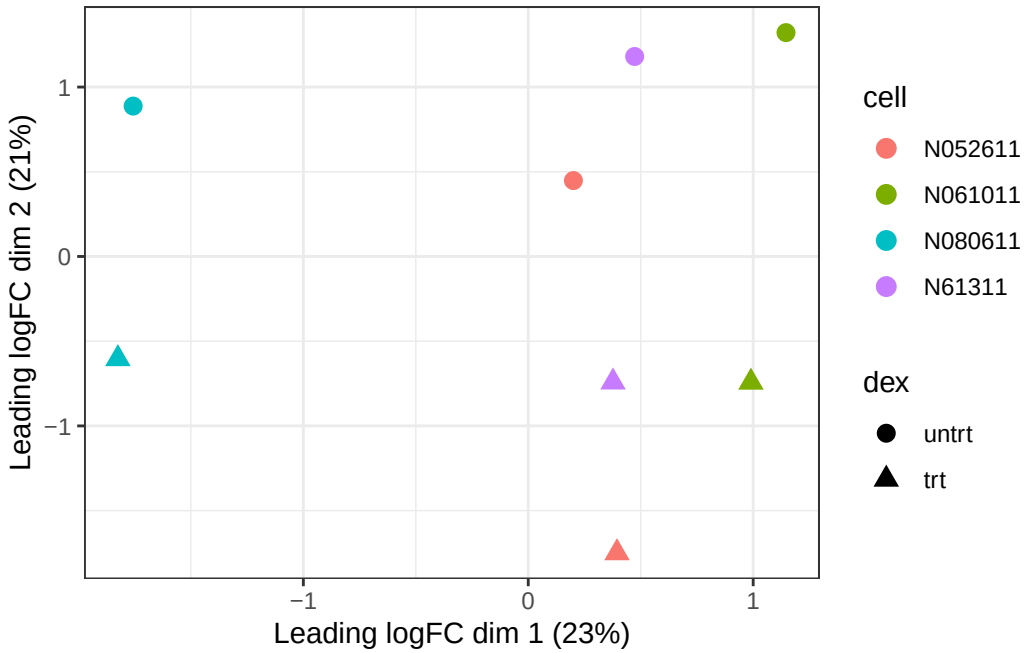
Filtering and normalisation

```
keep <- filterByExpr(dge, design)
dge <- dge[keep, , keep.lib.sizes = FALSE]
dge <- normLibSizes(dge)
nrow(dge)
```

```
[1] 18103
```

Exploratory data analysis

```
mds <- plotMDS(dge, top = 500, plot = FALSE)
tibble(dim1 = mds$x, dim2 = mds$y, cell = dge$samples$cell, dex = dge$samples$dex) |>
  ggplot(aes(dim1, dim2, color = cell, shape = dex)) +
  geom_point(size = 3) +
  xlab(paste0("Leading logFC dim 1 (", round(mds$var.explained[1] * 100), "%)")) +
  ylab(paste0("Leading logFC dim 2 (", round(mds$var.explained[2] * 100), "%)")) +
  theme_bw()
```



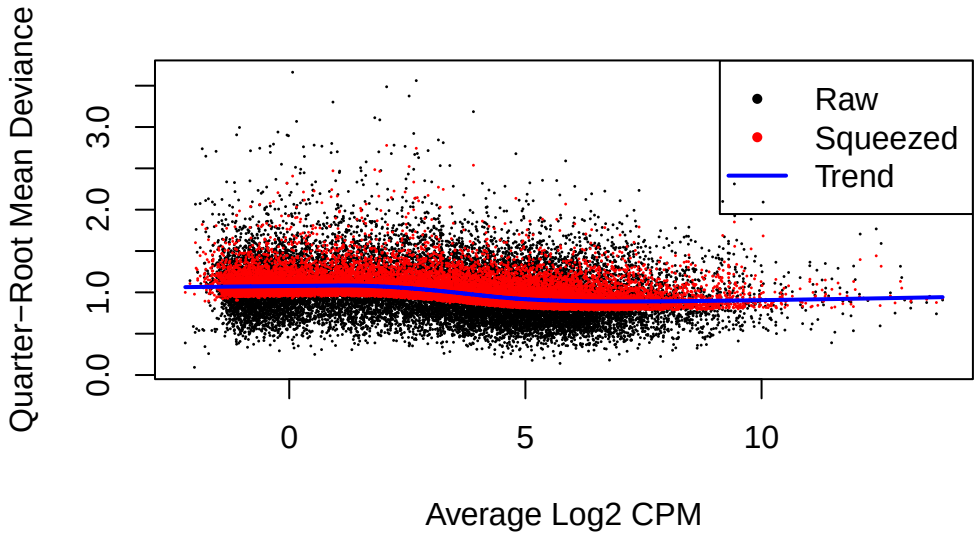
Question

Which sources of variability do the first two dimensions capture?

- The first dimension mainly separates cell line N080611 from the other cell lines: the cell line is an important source of variability, which confirms that we have to correct for it in the model.
- The second dimension separates the treated from the untreated samples: dexamethasone has a strong effect, which is consistent in all cell lines.

Model fit and inference

```
fit <- glmQLFit(dge, design)
plotQLDisp(fit)
```




```
qlf <- glmQLFTest(fit, coef = "dextrt")
summary(decideTests(qlf))
```

```
       dextrt
Down     1860
NotSig  14361
Up       1882
```

```
topTags(qlf, n = 10)$table |>
  dplyr::select(SYMBOL, logFC, logCPM, F, PValue, FDR)
```

```
                        SYMBOL     logFC   logCPM        F       PValue
ENSG00000168309.17      FAM107A  4.718963 2.822957 688.2611 6.150851e-10
ENSG00000109906.13       ZBTB16  6.087052 4.119388 865.7767 4.729581e-09
ENSG00000157214.13       STEAP2  1.987709 7.092884 791.6267 6.462766e-09
ENSG00000146250.6        PRSS35 -2.753374 3.822224 713.0864 9.560344e-09
ENSG00000250978.5  LOC101928819  5.771354 1.178493 290.2749 1.239010e-08
ENSG00000162614.18         NEXN  1.999143 7.983201 627.5350 1.539656e-08
ENSG00000120129.5         DUSP1  2.940487 7.246624 624.0632 1.571876e-08
ENSG00000189221.9          MAOA  3.375417 6.713138 620.0646 1.610092e-08
ENSG00000125148.6          MT2A  2.203431 7.360873 550.1616 2.515297e-08
```

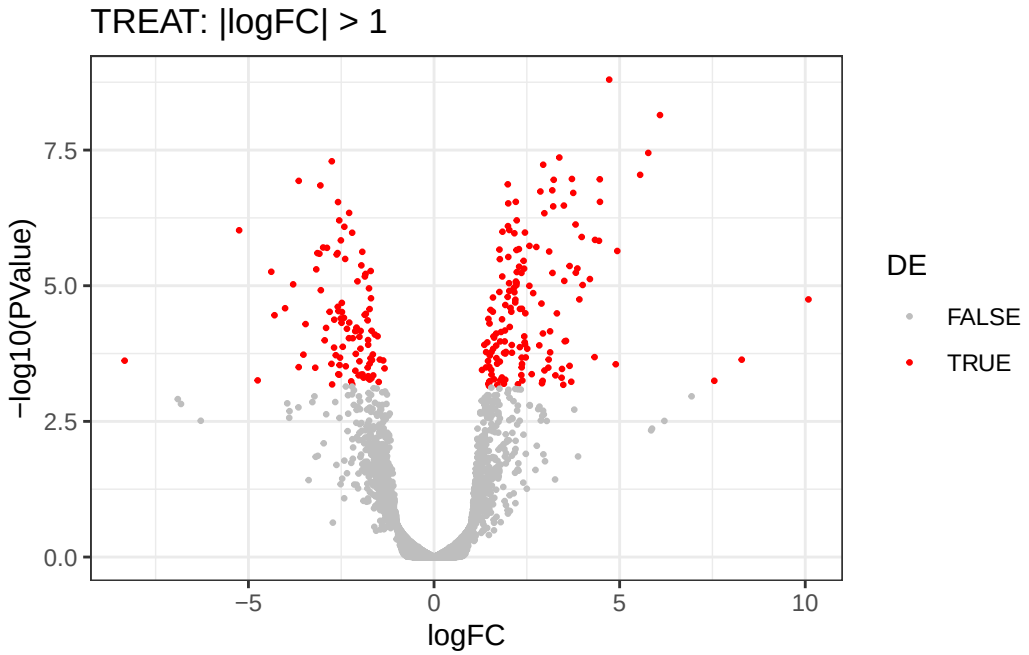
```
ENSG00000154930.14        ACSS1  1.844333 4.896571 506.2940 3.428101e-08
                              FDR
ENSG00000168309.17 1.113489e-05
ENSG00000109906.13 3.643436e-05
ENSG00000157214.13 3.643436e-05
ENSG00000146250.6  3.643436e-05
ENSG00000250978.5  3.643436e-05
ENSG00000162614.18 3.643436e-05
ENSG00000120129.5  3.643436e-05
ENSG00000189221.9  3.643436e-05
ENSG00000125148.6  4.561058e-05
ENSG00000154930.14 4.561058e-05
```

Thousands of genes are differentially expressed. We therefore use TREAT to focus on genes with a fold change that is significantly larger than 2.

```
treat <- glmTreat(fit, coef = "dextrt", lfc = 1)
summary(decideTests(treat))
```

```
       dextrt
Down      101
NotSig  17849
Up        153
```

```
topTags(treat, n = Inf)$table |>
  mutate(DE = FDR < 0.05) |>
  ggplot(aes(logFC, -log10(PValue), color = DE)) +
  geom_point(size = 0.5) +
  scale_color_manual(values = c("grey", "red")) +
  ggtitle("TREAT: |logFC| > 1") +
  theme_bw()
```



Question

Look up the top genes. Are these known targets of glucocorticoids?

- The top genes include well-known glucocorticoid-responsive genes, e.g. *ZBTB16*, *FAM107A*, *DUSP1*, *MT2A* and *TSC22D3*. The latter encodes GILZ, the glucocorticoid-induced leucine zipper.
- These genes serve as a positive control for the workflow.

Part 2: differential transcript expression

Quantification uncertainty

We now analyse the counts of the individual transcripts. Reads that are compatible with several transcripts of the same gene are assigned with the EM algorithm, which makes the transcript counts more variable than the gene counts. `DGEListFromTximeta` estimates an overdispersion factor for every transcript from the Gibbs samples, and with `divide = TRUE` it returns the counts divided by this factor (Baldoni et al. 2024).

Quantification uncertainty is especially an issue for transcripts. Most ambiguous reads are compatible with several transcripts of the same gene: the assignment of such a read changes the transcript counts, but not their sum. The gene counts are therefore much less affected, which is

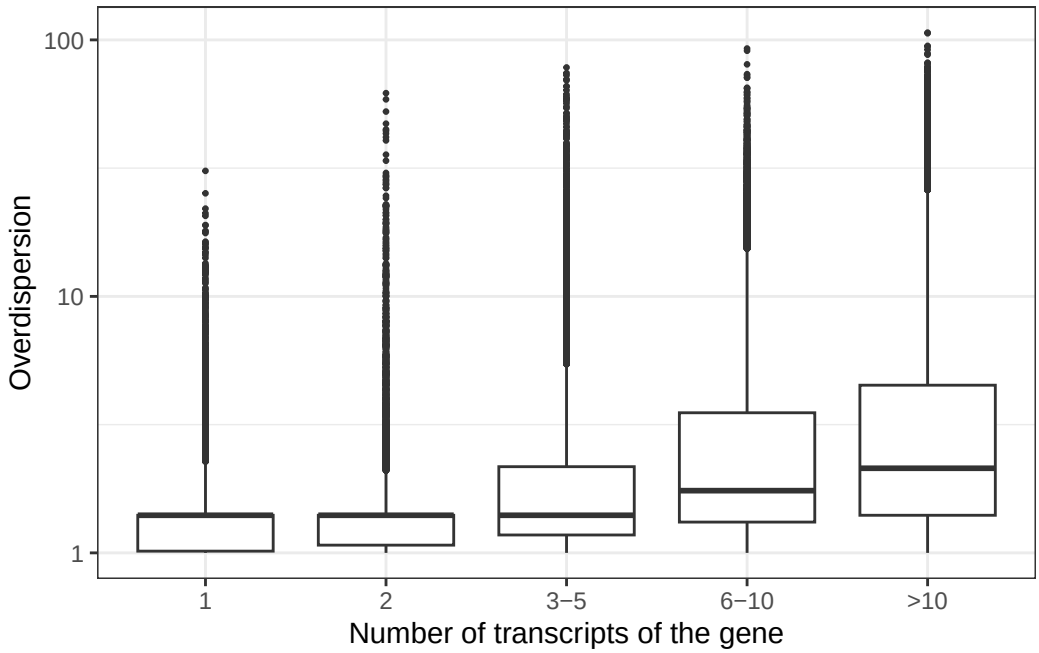
why we analysed the summed gene counts in Part 1 without accounting for the quantification uncertainty.

```
txRaw <- DGEListFromTximeta(st)
txDivided <- DGEListFromTximeta(st, divide = TRUE)
summary(txDivided$genes$Overdispersion)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.000   1.246   1.447   3.238   3.026 106.734
```

```
txAnnotation <- tibble(
  tx = txDivided$genes$tx_name,
  gene = sapply(txDivided$genes$gene_id, `[`, 1),
  Overdispersion = txDivided$genes$Overdispersion
) |>
  group_by(gene) |>
  mutate(nTx = n()) |>
  ungroup()

txAnnotation |>
  mutate(nTx = cut(nTx, c(0, 1, 2, 5, 10, Inf), labels = c("1", "2", "3-5", "6-10", ">10"))) |>
  ggplot(aes(nTx, Overdispersion)) +
  geom_boxplot(outlier.size = 0.5) +
  scale_y_log10() +
  xlab("Number of transcripts of the gene") +
  theme_bw()
```



Question

How does the overdispersion depend on the number of transcripts of the gene? Explain.

- The overdispersion increases with the number of transcripts of the gene: the more transcripts a gene has, the more reads are compatible with several transcripts, and the larger the uncertainty on the transcript counts.
- Transcripts of genes with one or two transcripts have the smallest overdispersions. They are not all equal to 1, because reads can also be ambiguous between genes, e.g. between paralogous genes or between a gene and its pseudogenes.

Analysis of raw and divided counts

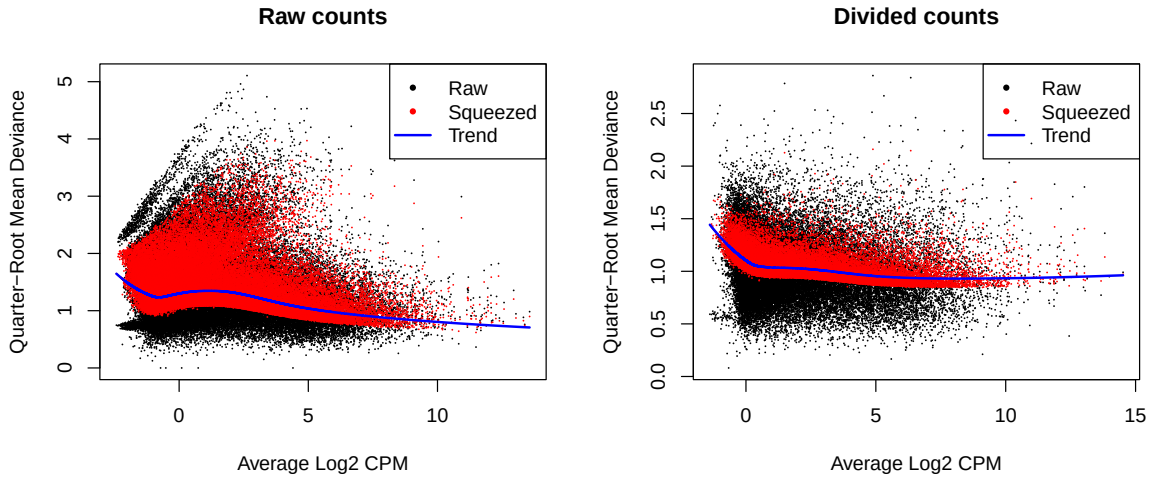
We analyse the raw and the divided transcript counts with the same edgeR pipeline.

```
dteAnalysis <- function(y) {
  y$genes <- data.frame(tx = y$genes$tx_name,
                        gene = sapply(y$genes$gene_id, `[`, 1),
                        Overdispersion = y$genes$Overdispersion,
                        row.names = rownames(y))
  keep <- filterByExpr(y, design)
  y <- y[keep, , keep.lib.sizes = FALSE]
```

```
  y <- normLibSizes(y)
  fit <- glmQLFit(y, design)
  list(y = y, fit = fit, test = glmQLFTest(fit, coef = "dextrt"))
}
dteRaw <- dteAnalysis(txRaw)
dteDivided <- dteAnalysis(txDivided)
c(raw = nrow(dteRaw$y), divided = nrow(dteDivided$y))
```

```
    raw divided
  58541   35448
```

```
par(mfrow = c(1, 2))
plotQLDisp(dteRaw$fit, main = "Raw counts")
plotQLDisp(dteDivided$fit, main = "Divided counts")
```




```
c(raw = median(dteRaw$fit$s2.post), divided = median(dteDivided$fit$s2.post))
```

```
     raw  divided
2.839723 1.137553
```

```
cbind(raw = summary(decideTests(dteRaw$test)), divided = summary(decideTests(dteDivided$test)
```

```
       dextrt dextrt
```

```
Down      335   1780
NotSig  57604  31292
Up        602   2376
```

> Question
>
> - Why are fewer transcripts retained after filtering when we use the divided counts?
> - Why are the QL dispersions of the raw counts much larger than one?
> - Why do we find many more differentially expressed transcripts with the divided counts?

- Dividing the counts by the overdispersion makes them smaller. Transcripts with a highly uncertain count then often fall below the filtering threshold: they carry little information.
- The NB model of edgeR does not capture the extra variability due to the read-to-transcript ambiguity. With the raw counts, this variability ends up in the QL dispersions, which are inflated for all transcripts: the empirical Bayes moderation shrinks them towards a trend that is too high.
- After dividing, the counts have the usual mean-variance relation again. The QL dispersions are centred around one, and transcripts with an unambiguous count are no longer penalised for the uncertainty of the other transcripts: the power increases.

Comparison with the gene-level results

```
dteResults <- topTags(dteDivided$test, n = Inf)$table
dgeSig <- rownames(dge)[decideTests(qlf) != 0]
dteGenes <- unique(dteResults$gene[dteResults$FDR < 0.05])
c(DTEgenes = length(dteGenes),
  alsoDGE = sum(dteGenes %in% dgeSig),
  notDGE = sum(!dteGenes %in% dgeSig))
```

```
DTEgenes  alsoDGE   notDGE
    3088     2514      574
```

> Question
>
> Some genes have a differentially expressed transcript, but are not differentially expressed at the gene level. How is this possible?

- A gene can switch between transcripts without changing its total expression: the up- and down-regulation of its transcripts cancel out at the gene level.

- A small transcript of a highly expressed gene can change without having a noticeable effect on the gene total.
- Both situations point to differential transcript usage, which we study next.

Part 3: differential transcript usage

Only genes with at least two transcripts can show DTU. We start from the filtered divided transcript counts.

```
yTx <- dteDivided$y
multiTxGenes <- yTx$genes |>
  dplyr::count(gene) |>
  filter(n > 1) |>
  pull(gene)
dtu <- yTx[yTx$genes$gene %in% multiTxGenes, ]
c(genes = length(multiTxGenes), transcripts = nrow(dtu))
```

```
genes transcripts
 8356       27564
```

Juggling offsets

We replace the offset by the log of the total count of the gene, so that the model parameters describe the log usage of the transcripts (Segers et al. 2023).

We model the divided counts $\tilde{Y}_{gti} = Y_{gti}/c_t$, with c_t the overdispersion of transcript t, so that the quantification uncertainty of the transcripts is accounted for. We model the counts conditionally on the observed gene total $Y_{g.i}$. Given the gene total, their mean is the usage times the gene total, divided by c_t:

$$E[\tilde{Y}_{gti} \mid Y_{g.i}] = \frac{\pi_{gti}\, Y_{g.i}}{c_t} \quad \Rightarrow \quad \text{offset} = \log Y_{g.i} - \log c_t,$$

with π_{gti} the usage of transcript t, i.e. the fraction of the fragments of gene g that come from transcript t.

- The gene total $Y_{g.i}$ is computed from the **undivided** counts.
- The undivided gene total is hardly affected by the quantification uncertainty: the ambiguity between transcripts of the same gene cancels out in the sum.

- The term $-\log c_t$ is the same in all samples. It only changes the intercept of the transcript, not the log fold changes.

The undivided counts are the divided counts multiplied by the overdispersion.

```
undivided <- dtu$counts * dtu$genes$Overdispersion
geneTotals <- rowsum(undivided, dtu$genes$gene)[dtu$genes$gene, ]
dtu$offset <- log(pmax(geneTotals, 1)) - log(dtu$genes$Overdispersion)

fitDtu <- glmQLFit(dtu, design)
qlfDtu <- glmQLFTest(fitDtu, coef = "dextrt")
summary(decideTests(qlfDtu))
```

```
       dextrt
Down       41
NotSig  27484
Up         39
```

```
fitDtu$dispersion
```

```
[1] 0.001212975
```

> Question
>
> - Why do we not need TMM normalisation in this model?
> - Interpret the estimated global NB dispersion.

- The gene total already accounts for the sequencing depth and the composition of the library.
- The global NB dispersion is almost zero: given the gene total, the biological variability of the expression of the gene cancels out in the usages, and the model reduces to a quasi-Poisson model on the usages.

diffSplice

`diffSplice` tests whether the log fold change of a transcript differs from the log fold change of its gene (Baldoni et al. 2025). `diffSplice` is a generic function: for an edgeR fit from `glmQLFit` it calls the method `diffSplice.DGEGLM` (see `?diffSplice.DGEGLM`).

```
fitTx <- dteDivided$fit
ds <- diffSplice(fitTx[fitTx$genes$gene %in% multiTxGenes, ],
                 coef = "dextrt", geneid = "gene", exonid = "tx", verbose = FALSE)
c(transcripts = sum(p.adjust(ds$p.value, "BH") < 0.05),
  genesF = sum(p.adjust(ds$gene.F.p.value, "BH") < 0.05),
  genesSimes = sum(p.adjust(ds$gene.simes.p.value, "BH") < 0.05))
```

```
transcripts      genesF  genesSimes
        212         119         107
```

```
table(F = p.adjust(ds$gene.F.p.value, "BH") < 0.05,
      Simes = p.adjust(ds$gene.simes.p.value, "BH") < 0.05)
```

```
       Simes
F       FALSE TRUE
  FALSE  8229    8
  TRUE     20   99
```

> Question
>
> Compare the genes that are found with the gene-level F-test and with the Simes test. When do you expect each test to be most powerful?

- Both tests largely agree, but each test also finds genes that the other test misses.
- The F-test pools the evidence of all transcripts of the gene. It is most powerful when several transcripts change, but the evidence of a single strongly changing transcript is diluted when the gene has many transcripts that do not change.
- The Simes test is driven by the smallest transcript-level p-values of the gene and is most powerful when one or a few transcripts stand out.

Stage-wise testing

stageR (Van den Berge et al. 2017) needs a gene-level p-value for the screening stage and transcript-level p-values for the confirmation stage. We perform two stage-wise analyses, each of which uses a single DTU method for both stages:

1. **diffSplice**: the gene-level Simes p-values of `diffSplice` for screening and its transcript-level tests for confirmation.
2. **Offset model**: the Simes method applied to the transcript-level p-values of the offset model for screening, and the same transcript-level p-values for confirmation.

The Simes p-value of a gene with n transcripts combines the ordered transcript-level p-values $p_{(1)} \leq ... \leq p_{(n)}$ into $\min_k n\, p_{(k)}/k$. Other aggregation methods exist, e.g. the minimum p-value with a Šidák correction (DEXSeq's `perGeneQValue`, which is used in the satuRn workflow) or the harmonic mean p-value (Wilson 2019), which Gilis et al. (2025) used to combine dependent tests.

```
simes <- function(p) min(sort(p) * length(p) / seq_along(p))

stageWiseDtu <- function(pScreen, pTranscript, tx, gene) {
  pConfirmation <- matrix(pTranscript, ncol = 1, dimnames = list(tx, "transcript"))
  stageRObj <- stageRTx(pScreen = pScreen, pConfirmation = pConfirmation,
                        tx2gene = data.frame(transcript = tx, gene = gene))
  stageRObj <- stageWiseAdjustment(stageRObj, method = "dtu", alpha = 0.05)
  getAdjustedPValues(stageRObj, order = TRUE, onlySignificantGenes = TRUE) |>
    mutate(symbol = mapIds(org.Hs.eg.db, sub("\\..*", "", geneID), "SYMBOL", "ENSEMBL"))
}

padjDs <- stageWiseDtu(
  pScreen = setNames(as.vector(ds$gene.simes.p.value), ds$gene.genes$gene),
  pTranscript = as.vector(ds$p.value), tx = ds$genes$tx, gene = ds$genes$gene
)
padjOffset <- stageWiseDtu(
  pScreen = c(tapply(qlfDtu$table$PValue, dtu$genes$gene, simes)),
  pTranscript = qlfDtu$table$PValue, tx = dtu$genes$tx, gene = dtu$genes$gene
)
head(padjDs, 10)
```

```
               geneID              txID         gene   transcript  symbol
1  ENSG00000157514.16 ENST00000372390.8 4.859587e-09 2.270835e-11 TSC22D3
2  ENSG00000157514.16 ENST00000372397.6 4.859587e-09 9.203644e-07 TSC22D3
3  ENSG00000157514.16 ENST00000486554.1 4.859587e-09 1.137615e-01 TSC22D3
4  ENSG00000157514.16 ENST00000372383.8 4.859587e-09 1.000000e+00 TSC22D3
5  ENSG00000065534.18 ENST00000578202.1 4.100412e-07 1.226291e-08    MYLK
6  ENSG00000065534.18 ENST00000464489.5 4.100412e-07 9.001532e-02    MYLK
7  ENSG00000065534.18 ENST00000360304.7 4.100412e-07 1.600118e-01    MYLK
8  ENSG00000065534.18 ENST00000506361.5 4.100412e-07 3.369508e-01    MYLK
9  ENSG00000065534.18 ENST00000514623.1 4.100412e-07 1.000000e+00    MYLK
10 ENSG00000065534.18 ENST00000360772.7 4.100412e-07 1.000000e+00    MYLK
```

```
head(padjOffset, 10)
```

```
               geneID              txID         gene   transcript symbol
```

```
1  ENSG00000109861.15  ENST00000524463.5 0.004244306 0.0001699235   CTSC
2  ENSG00000109861.15  ENST00000227266.9 0.004244306 0.0031293154   CTSC
3  ENSG00000109861.15  ENST00000529974.1 0.004244306 0.0070451683   CTSC
4  ENSG00000109861.15  ENST00000533897.1 0.004244306 1.0000000000   CTSC
5  ENSG00000140545.14  ENST00000617199.1 0.004244306 0.0003420320  MFGE8
6  ENSG00000140545.14 ENST00000268151.11 0.004244306 1.0000000000  MFGE8
7  ENSG00000140545.14  ENST00000559259.1 0.004244306 1.0000000000  MFGE8
8  ENSG00000140545.14  ENST00000560937.1 0.004244306 1.0000000000  MFGE8
9  ENSG00000140545.14  ENST00000557944.1 0.004244306 1.0000000000  MFGE8
10 ENSG00000140545.14  ENST00000613965.1 0.004244306 1.0000000000  MFGE8
```

```r
rbind(
  diffSplice = c(genes = length(unique(padjDs$geneID)), transcripts = sum(padjDs$transcript <
  offset = c(genes = length(unique(padjOffset$geneID)), transcripts = sum(padjOffset$transcri
  both = c(genes = length(intersect(padjDs$geneID, padjOffset$geneID)),
           transcripts = length(intersect(padjDs$txID[padjDs$transcript < 0.05],
                                          padjOffset$txID[padjOffset$transcript < 0.05])))
)
```

```
           genes transcripts
diffSplice   107         164
offset        42          56
both          35          42
```

> **Question**
>
> How does stage-wise testing control the error rates? Why is this attractive for DTU analyses?

- The screening stage controls the FDR at the gene level. In the confirmation stage, the transcripts of the significant genes are tested with a family-wise error rate correction within each gene.
- Biologists are mostly interested in the genes with DTU and in the transcripts that drive it. Stage-wise testing answers both questions while controlling the overall FDR, and the gene-level screening reduces the multiple testing burden of the transcript-level tests.

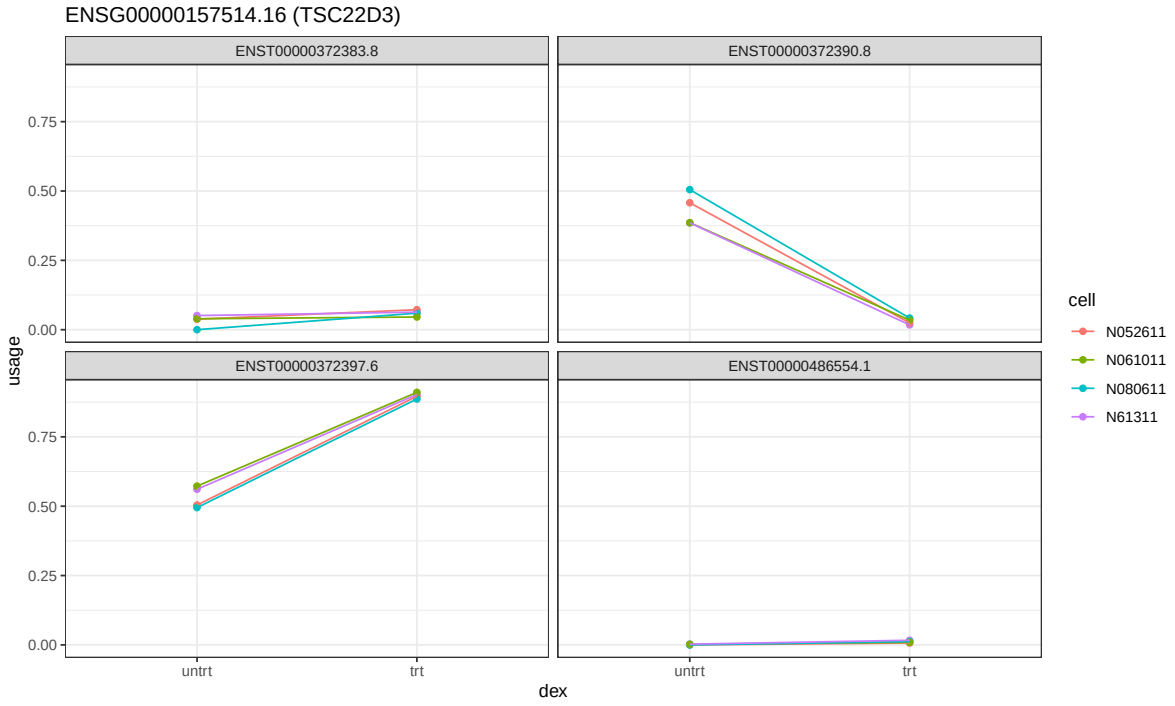
> **Question**
>
> Compare the results of both stage-wise analyses. Which approach is the most powerful here? Can you think of reasons why?

- `diffSplice` finds many more genes and transcripts than the offset model. Most genes that are found with the offset model are also found with `diffSplice`.
- Both methods test a different quantity. `diffSplice` compares the log fold change of each transcript with that of the gene as a whole in the transcript-level DTE model, in which the biological variability of the expression is modelled for every transcript. The offset model plugs the observed gene total in as a known offset, so that it models the usage directly, and it estimates a separate QL dispersion for the usage of every transcript.
- In the simulation study of Baldoni et al. (2025), `diffSplice` on divided counts with Simes p-values was the most powerful method, while it controlled the FDR. On the macrophage data of the lecture, however, the offset model finds more genes than `diffSplice`: which method is the most powerful depends on the data.

Visualisation

We plot the usage of the transcripts of the top gene of the `diffSplice` analysis.

```
plotUsage <- function(geneId) {
  sel <- dtu$genes$gene == geneId
  usage <- undivided[sel, , drop = FALSE] / geneTotals[sel, , drop = FALSE]
  rownames(usage) <- dtu$genes$tx[sel]
  usage |>
    as_tibble(rownames = "tx") |>
    pivot_longer(-tx, names_to = "names", values_to = "usage") |>
    left_join(coldata, by = "names") |>
    ggplot(aes(dex, usage, group = cell, color = cell)) +
    geom_point() +
    geom_line() +
    facet_wrap(~ tx) +
    ggtitle(paste0(geneId, " (", mapIds(org.Hs.eg.db, sub("\\..*", "", geneId), "SYMBOL", "E
    theme_bw()
}
plotUsage(padjDs$geneID[1])
```



> **Question**
>
> - Which transcripts of the top gene change their usage?
> - Is this gene also differentially expressed at the gene level?

```
topTags(qlf, n = Inf)$table[padjDs$geneID[1], c("SYMBOL", "logFC", "FDR")]
```

```
                      SYMBOL   logFC          FDR
ENSG00000157514.16 TSC22D3 3.19209 0.0002292073
```

- The top gene is *TSC22D3*, which encodes GILZ. Its usage shifts consistently in all cell lines: transcript ENST00000372397 accounts for about half of the expression of the gene in the untreated cells and for about 90% after the treatment, while the usage of ENST00000372390 drops from about 40-50% to almost zero.
- Dexamethasone also strongly induces the gene as a whole: *TSC22D3* is up-regulated about 9-fold ($\log_2$ FC ≈ 3.2) at the gene level. Dexamethasone thus not only induces the gene, but also shifts the usage between its transcripts.
- A gene can thus show both DGE and DTU: the questions are complementary.

```r
dtuGenes <- unique(padjDs$geneID)
c(DTUgenes = length(dtuGenes), alsoDGE = sum(dtuGenes %in% dgeSig))
```

```
DTUgenes  alsoDGE
     107       65
```

Fragment usage or molecular usage?

The counts are numbers of fragments. Let M_{gti} be the (unobserved) number of molecules of transcript t of gene g in the library of sample i, $M_{g.i} = \sum_{t'} M_{gt'i}$, and $M_{g\cdot i}$ the numbers of molecules of all transcripts of the gene. The **molecular usage** is the fraction of the molecules of the gene that are transcript t, $\rho_{gti} = M_{gti}/M_{g.i}$.

A transcript produces fragments in proportion to its number of molecules times its effective length l_{ti}:

$$E[Y_{gti} \mid M_{g\cdot i}] = k_i \, M_{gti} \, l_{ti} = \rho_{gti} \; l_{ti} \; k_i M_{g.i},$$

with k_i the expected number of sequenced fragments per molecule and per base in sample i. It depends on the sequencing depth and on the composition of the library, but it is the same for all transcripts in the sample. Dividing by the effective length and summing over the transcripts of the gene gives $\sum_{t'} E[Y_{gt'i} \mid M_{g\cdot i}]/l_{t'i} = k_i M_{g.i}$, which we estimate by the length-normalised gene total $\sum_{t'} Y_{gt'i}/l_{t'i}$. Hence

$$\log \mu_{gti} = \log \rho_{gti} + \log l_{ti} + \log \sum_{t'} \frac{Y_{gt'i}}{l_{t'i}} \; :$$

with $\log l_{ti} + \log \sum_{t'} Y_{gt'i}/l_{t'i}$ as offset, i.e. the length-normalised gene total multiplied by the effective length of the transcript, the model parameters describe the log molecular usage, $\log \rho_{gti} = \mathbf{x}_i^T \beta_{gt}$.

With the gene total $Y_{g.i}$ as offset, we instead model the **fragment usage**: the expected fraction of the fragments of the gene that come from transcript t,

$$\pi_{gti} = \frac{\rho_{gti} \, l_{ti}}{\sum_{t'} \rho_{gt'i} \, l_{t'i}}.$$

It also changes when the gene shifts towards longer transcripts. For example, consider a gene with two transcripts of 1000 and 3000 bp. In the control, both transcripts have the same number of molecules, but the long transcript produces three times as many fragments:

	ρ_{g1i}	ρ_{g2i}	π_{g1i}	π_{g2i}
control	0.5	0.5	0.25	0.75
treatment	0.4	0.6	0.18	0.82

- Under treatment, the molecular usage of transcript 1 decreases with a factor 0.8, its fragment usage with a factor 0.73: the longer transcript, whose molecular usage increases, takes an even larger share of the fragments.
- If a gene shifts towards longer transcripts, the fragment usage of a transcript whose molecular usage does not change, also decreases. Tests on the fragment usage can thus flag transcripts that do not change themselves.

This is the offset counterpart of `countsFromAbundance = "dtuScaledTPM"` in tximport, which Love et al. (2018) recommend for DTU analyses.

- Because salmon's TPM is proportional to Y_{gti}/l_{ti}, the estimated molecular usage $\hat{\rho}_{gti} = (Y_{gti}/l_{ti})/\sum_{t'} Y_{gt'i}/l_{t'i}$ is the TPM of the transcript divided by the TPM of the gene.
- As for the gene total, we compute the sum with the undivided counts and subtract $\log c_t$ for the divided counts.
- We use the sample-specific effective lengths of salmon, which also account for the fragment length distribution and the sequence and GC bias of each sample.

```
effLength <- assay(st, "length")[rownames(dtu), ]
lengthNormTotals <- rowsum(undivided / effLength, dtu$genes$gene)[dtu$genes$gene, ]

dtuLength <- dtu
# as for the gene totals, the offset of a gene with zero counts in a sample is set to 0
dtuLength$offset <- log(effLength) + log(pmax(lengthNormTotals, 1 / effLength)) -
  log(dtu$genes$Overdispersion)

fitDtuLength <- glmQLFit(dtuLength, design)
qlfDtuLength <- glmQLFTest(fitDtuLength, coef = "dextrt")
```

Why do both analyses give different log fold changes? Dividing the numerator and denominator of the fragment usage by $M_{g.i}$ links both usages through the average transcript length of the gene, weighted by the number of molecules of each transcript:

$$\pi_{gti} = \rho_{gti}\,\frac{l_{ti}}{\bar{L}_{gi}}, \qquad \bar{L}_{gi} = \sum_{t'} \rho_{gt'i}\, l_{t'i} \quad \Rightarrow \quad \log\rho_{gti} - \log\pi_{gti} = \log\bar{L}_{gi} - \log l_{ti}.$$

The effective length l_{ti} hardly changes between conditions, so the difference between both log fold changes of a transcript should equal the change in the log2 average transcript length of

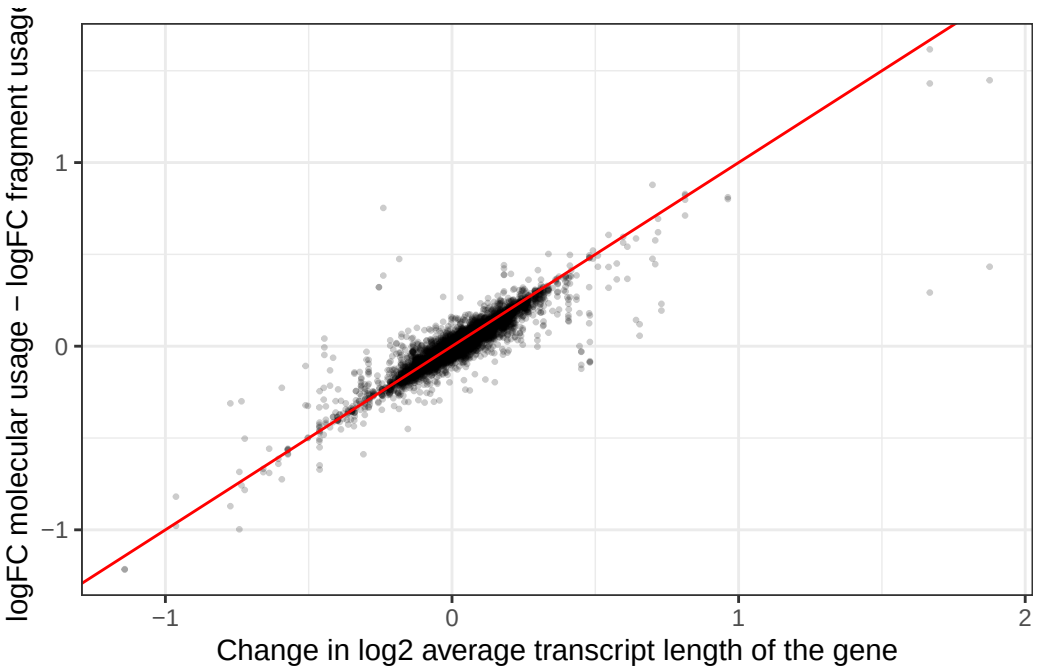
its gene, which is the same for all transcripts of the gene. We estimate the average transcript length with

$$\hat{\bar{L}}_{gi} = \sum_{t'} \hat{\rho}_{gt'i}\, l_{t'i} = \frac{Y_{g.i}}{\sum_{t'} Y_{gt'i}/l_{t'i}},$$

which is exactly the TPM-weighted average transcript length that tximport computes at the gene level. The hat denotes an estimate from the observed counts.

```
avgLength <- rowsum(undivided, dtu$genes$gene) / rowsum(undivided / effLength, dtu$genes$gene
deltaAvgLength <- rowMeans(log2(avgLength[, coldata$dex == "trt"])) -
  rowMeans(log2(avgLength[, coldata$dex == "untrt"]))

tibble(
  deltaAvgLength = deltaAvgLength[dtu$genes$gene],
  logFCdifference = qlfDtuLength$table$logFC - qlfDtu$table$logFC
) |>
  ggplot(aes(deltaAvgLength, logFCdifference)) +
  geom_point(alpha = 0.2, size = 0.5) +
  geom_abline(intercept = 0, slope = 1, col = "red") +
  xlab("Change in log2 average transcript length of the gene") +
  ylab("logFC molecular usage - logFC fragment usage") +
  theme_bw()
```



The points scatter closely around the identity line. Small deviations arise because the effective lengths differ slightly between samples and because the log fold changes are estimated with a GLM rather than computed from the average usages.

We compare the transcripts that are significant in both analyses, first without and then with stage-wise testing.

```
table(fragment = p.adjust(qlfDtu$table$PValue, "BH") < 0.05,
      molecular = p.adjust(qlfDtuLength$table$PValue, "BH") < 0.05)
```

```
        molecular
fragment FALSE  TRUE
   FALSE 27474    10
   TRUE     32    48
```

For the stage-wise analysis of the molecular usage, we again screen the genes with the Simes p-values of the transcript-level tests of the same model. The null hypothesis of the screening stage is the same on both scales: no DTU in terms of molecules is equivalent to no DTU in terms of fragments. The Simes p-values, however, combine different transcript-level tests, so the selected genes can differ.

```
padjLength <- stageWiseDtu(
  pScreen = c(tapply(qlfDtuLength$table$PValue, dtu$genes$gene, simes)),
  pTranscript = qlfDtuLength$table$PValue, tx = dtu$genes$tx, gene = dtu$genes$gene
)

stageWise <- full_join(
  padjOffset |> dplyr::select(txID, geneID, symbol, fragment = transcript),
  padjLength |> dplyr::select(txID, geneID, symbol, molecular = transcript),
  by = c("txID", "geneID", "symbol")
)
rbind(
  fragment = c(genes = length(unique(padjOffset$geneID)), transcripts = sum(padjOffset$transc
  molecular = c(genes = length(unique(padjLength$geneID)), transcripts = sum(padjLength$trans
  both = c(genes = length(intersect(padjOffset$geneID, padjLength$geneID)),
           transcripts = sum(stageWise$fragment < 0.05 & stageWise$molecular < 0.05, na.rm =
)
```

```
          genes transcripts
fragment     42          56
molecular    39          55
both         32          40
```

The transcripts that are significant in only one of both analyses (`NA`: the gene is not selected in the screening stage):

```
stageWise |>
  filter(xor(fragment < 0.05 & !is.na(fragment), molecular < 0.05 & !is.na(molecular))) |>
  mutate(
    length = round(rowMeans(effLength[txID, ])),
    logFCfragment = qlfDtu$table[txID, "logFC"],
    logFCmolecular = qlfDtuLength$table[txID, "logFC"]
  ) |>
  dplyr::select(symbol, txID, length, logFCfragment, fragment, logFCmolecular, molecular) |>
  knitr::kable(digits = 3)
```

symbol	txID	length	logFCfragment	fragment	logFCmolecular	molecular
NLRP1	ENST00000568641.1	1186	-0.877	0.000	-0.397	NA
CCDC80	ENST00000439685.6	3648	-0.169	0.018	-0.043	1.000
NAV2	ENST00000396085.5	11280	-0.765	0.049	-0.749	0.071
DCN	ENST00000547568.6	697	-0.364	0.037	-0.336	0.055
CABIN1	ENST00000398319.6	7727	-0.695	0.146	-1.253	0.019
CD47	ENST00000471694.1	5978	-0.604	0.005	-0.461	NA
CD47	ENST00000355354.12	4361	0.193	0.034	0.325	NA
CD55	ENST00000367063.6	1525	-0.659	0.007	-0.601	NA
NAP1L1	ENST00000261182.12	3661	0.269	0.013	0.283	NA
TMSB4X	ENST00000380636.1	1443	-0.350	0.006	-0.357	NA
MYLK	ENST00000360304.7	8027	-0.312	0.228	-0.498	0.004
LIPT1	ENST00000480170.1	406	2.075	0.000	1.096	NA
LIPT1	ENST00000393473.6	1214	-2.435	0.000	-3.255	NA
CTSA	ENST00000646241.2	1759	0.114	0.023	0.128	NA
ZRANB2	ENST00000477096.1	2746	0.284	0.022	0.347	NA
FST	ENST00000504226.5	580	0.894	0.023	0.698	NA
SBF2	ENST00000525040.5	2073	0.802	0.030	0.445	1.000
SBF2	ENST00000256190.12	7381	-0.150	0.298	-0.499	0.001
ZNF451	ENST00000370708.8	8253	0.867	0.016	0.925	NA
SLC7A14	ENST00000231706.5	9624	-0.139	NA	-1.356	0.000
SLC7A14	ENST00000466168.1	339	1.981	NA	0.767	0.000
SAMD12	ENST00000409003.5	8320	-1.149	NA	-1.610	0.000
SAMD12	ENST00000445741.6	1361	0.563	NA	0.135	0.000
CNOT7	ENST00000361272.8	5901	0.122	NA	0.377	0.018
IFT27	ENST00000471809.5	6474	-0.151	NA	-0.381	0.000
IFT27	ENST00000433985.6	881	0.352	NA	0.120	0.000

symbol	txID	length	logFCfragment	fragment	logFCmolecular	molecular
ACTG1	ENST00000576544.5	1644	-9.473	NA	-9.374	0.029
KPNA4	ENST00000334256.8	7937	0.028	NA	0.410	0.000
KPNA4	ENST00000483437.1	366	-1.177	NA	-0.797	0.000
EXOSC9	ENST00000513654.5	3308	-0.533	NA	-0.726	0.000
EXOSC9	ENST00000243498.9	1377	0.396	NA	0.226	0.000

We look at *NLRP1* in more detail.

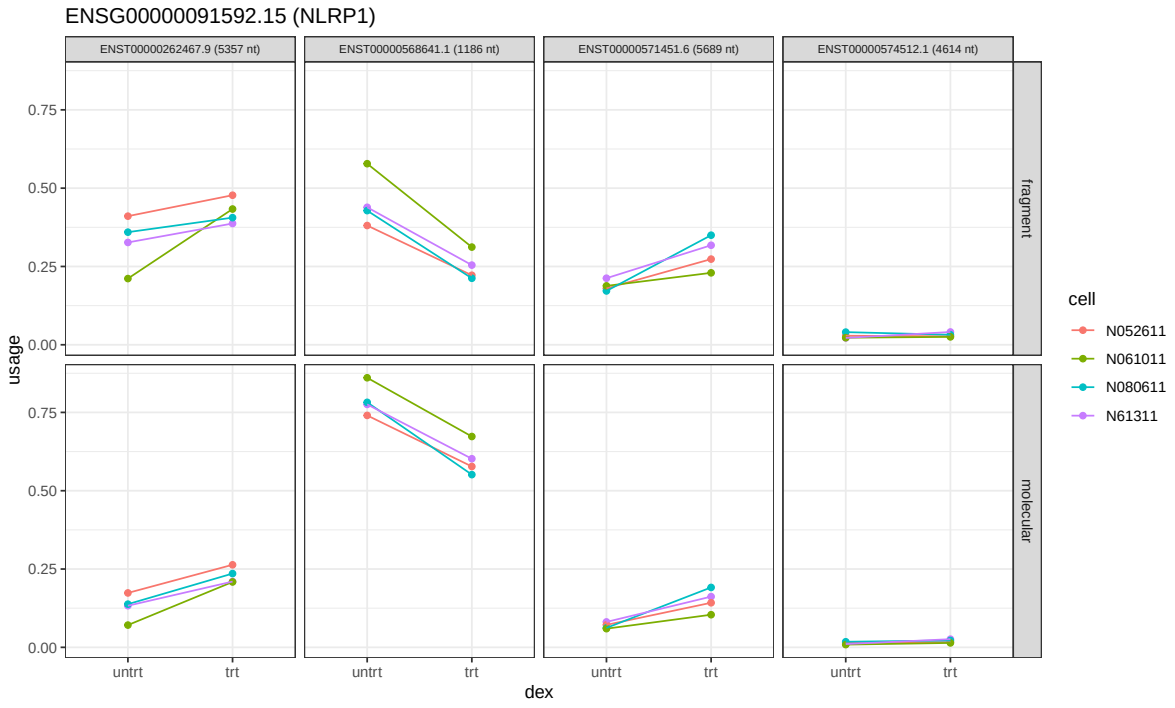
```
nlrp1 <- dtu$genes$gene == "ENSG00000091592.15"
data.frame(
  length = round(rowMeans(effLength[nlrp1, ])),
  logFCfragment = qlfDtu$table$logFC[nlrp1],
  pFragment = qlfDtu$table$PValue[nlrp1],
  logFCmolecular = qlfDtuLength$table$logFC[nlrp1],
  pMolecular = qlfDtuLength$table$PValue[nlrp1],
  row.names = dtu$genes$tx[nlrp1]
) |>
  knitr::kable(digits = 4)
```

	length	logFCfragment	pFragment	logFCmolecular	pMolecular
ENST00000568641.1	1186	-0.8769	0.0000	-0.3973	0.0001
ENST00000262467.9	5357	0.3820	0.0292	0.8671	0.0006
ENST00000574512.1	4614	0.1127	0.6241	0.6078	0.0197
ENST00000571451.6	5689	0.6478	0.0058	1.1289	0.0003

We plot the fragment and the molecular usage of its transcripts.

```
plotUsageScales <- function(geneId) {
  sel <- dtu$genes$gene == geneId
  counts <- undivided[sel, , drop = FALSE]
  molecules <- counts / effLength[sel, , drop = FALSE]
  bind_rows(
    fragment = as_tibble(t(t(counts) / colSums(counts)), rownames = "tx"),
    molecular = as_tibble(t(t(molecules) / colSums(molecules)), rownames = "tx"),
    .id = "scale"
```

```
  ) |>
    mutate(tx = paste0(dtu$genes$tx[sel][match(tx, rownames(counts))],
                       " (", round(rowMeans(effLength[tx, ])), " nt)")) |>
    pivot_longer(-c(scale, tx), names_to = "names", values_to = "usage") |>
    left_join(coldata, by = "names") |>
    ggplot(aes(dex, usage, group = cell, color = cell)) +
    geom_point() +
    geom_line() +
    facet_grid(scale ~ tx) +
    ggtitle(paste0(geneId, " (", mapIds(org.Hs.eg.db, sub("\\..*", "", geneId), "SYMBOL", "E
    theme_bw() +
    theme(strip.text.x = element_text(size = 7))
}
plotUsageScales("ENSG00000091592.15")
```



Question

- Why does the short transcript of *NLRP1* lose more usage on the fragment scale than on the molecular scale?
- Which analysis would you report to a biologist?

- Dexamethasone shifts the expression of *NLRP1* towards its long transcripts (about 5 kb). A long transcript produces about four times more fragments per molecule than the short transcript (1.2 kb). The fragment usage of the short transcript therefore decreases about twice as strongly on the log scale as its molecular usage: about half of its apparent decrease on the fragment scale is due to the shift to the long transcripts.
- On the fragment scale, the strong decrease of the abundant short transcript provides very strong evidence for DTU. On the molecular scale, the changes are more moderate and the evidence is spread over several transcripts. Although all transcript-level p-values of the molecular analysis are below 0.05, *NLRP1* just misses the screening stage of the molecular analysis.
- Biologists think in terms of molecules: the molecular usage, i.e. the fraction of the transcripts of the gene, is the quantity of interest. The length-corrected offset estimates exactly this quantity. Both analyses agree for most transcripts, but they differ for genes whose transcripts differ strongly in length.

Summary

```
tibble(
  analysis = c("DGE", "DGE (TREAT, |logFC| > 1)", "DTE (raw counts)", "DTE (divided counts)"
               "DTU diffSplice (stage-wise)", "DTU offset model, fragment usage (stage-wise)
               "DTU offset model, molecular usage (stage-wise)"),
  genes = c(sum(decideTests(qlf) != 0), sum(decideTests(treat) != 0), NA, NA,
            length(unique(padjDs$geneID)), length(unique(padjOffset$geneID)),
            length(unique(padjLength$geneID))),
  transcripts = c(NA, NA, sum(decideTests(dteRaw$test) != 0), sum(decideTests(dteDivided$tes
                  sum(padjDs$transcript < 0.05), sum(padjOffset$transcript < 0.05),
                  sum(padjLength$transcript < 0.05))
) |>
  knitr::kable()
```

analysis	genes	transcripts
DGE	3742	NA
DGE (TREAT, |logFC| > 1)	254	NA
DTE (raw counts)	NA	937
DTE (divided counts)	NA	4156
DTU diffSplice (stage-wise)	107	164
DTU offset model, fragment usage (stage-wise)	42	56
DTU offset model, molecular usage (stage-wise)	39	55

- DGE, DTE and DTU answer different biological questions and are complementary.
- Transcript counts are more uncertain than gene counts. Dividing them by the overdispersion estimated from the Gibbs samples restores the mean-variance relation of the NB model and greatly improves the power of the transcript-level analyses.
- DTU can be analysed with the standard edgeR pipeline by juggling the offsets, or with `diffSplice`.
- With the gene total as offset we model the fragment usage. Adding the effective transcript length to the offset models the molecular usage, which is the quantity of biological interest and avoids transcripts being flagged because other transcripts of the gene with a different length change.
- Stage-wise testing controls the FDR at the gene level and identifies the transcripts that drive the DTU. We used a single DTU method for both stages: `diffSplice` (Simes screening and transcript-level tests) or the offset model (Simes screening on its own transcript-level tests). On these data, `diffSplice` was much more powerful.

Further exercises

1. Repeat the DTE analysis with TREAT. How many transcripts change more than twofold?
2. Plot the usages of a few other DTU genes. Do you find genes that switch between transcripts without a change in their total expression?
3. Perform a stage-wise DTE analysis with stageR (`method = "dte"`): screen genes with the Simes test on the transcript-level DTE p-values and confirm the transcripts. Compare the number of genes with the DGE analysis.

References

Baldoni, Pedro L, Lizhong Chen, Mengbo Li, Yunshun Chen, and Gordon K Smyth. 2025. "Dividing Out Quantification Uncertainty Enables Assessment of Differential Transcript Usage with Limma and edgeR." *Nucleic Acids Research* 53 (22): gkaf1305. https://doi.org/10.1093/nar/gkaf1305.

Baldoni, Pedro L, Yunshun Chen, Soroor Hediyeh-zadeh, et al. 2024. "Dividing Out Quantification Uncertainty Allows Efficient Assessment of Differential Transcript Expression with edgeR." *Nucleic Acids Research* 52 (3): e13.

Gilis, Jeroen, Laura Perin, Milan Malfait, et al. 2025. "Differential Detection Workflows for Multi-Sample Single-Cell RNA-seq Data." *BMC Genomics* 26: 886. https://doi.org/10.1186/s12864-025-12102-x.

Himes, Blanca E, Xiaofeng Jiang, Peter Wagner, et al. 2014. "RNA-Seq Transcriptome Profiling Identifies CRISPLD2 as a Glucocorticoid Responsive Gene That Modulates Cytokine Function in Airway Smooth Muscle Cells." *PLoS ONE* 9 (6): e99625.

Love, Michael I., Charlotte Soneson, and Rob Patro. 2018. "Swimming Downstream: Statistical Analysis of Differential Transcript Usage Following Salmon Quantification." *F1000Research* 7: 952. https://doi.org/10.12688/f1000research.15398.3.

Segers, Alexandre, Jeroen Gilis, Mattias Van Heetvelde, Davide Risso, Elfride De Baere, and Lieven Clement. 2023. "Juggling Offsets Unlocks RNA-seq Tools for Fast and Scalable Differential Usage, Aberrant Splicing and Expression Retrieval." *bioRxiv*, 2023.06.29.547014.

Van den Berge, Koen, Charlotte Soneson, Mark D Robinson, and Lieven Clement. 2017. "stageR: A General Stage-Wise Method for Controlling the Gene-Level False Discovery Rate in Differential Expression and Differential Transcript Usage." *Genome Biology* 18: 151.

Wilson, Daniel J. 2019. "The Harmonic Mean p-Value for Combining Dependent Tests." *Proceedings of the National Academy of Sciences* 116 (4): 1195–200. https://doi.org/10.1073/pnas.1814092116.